POLYTECHNIQUE MONTRÉAL
affiliée à l'Université de Montréal

A lightweight canonicalization engine in C

WILLIAM ZHANG
Département de Electrical Engineering

Mémoire présenté en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
Electrical Engineering

July 2026

© William Zhang, 2026.

POLYTECHNIQUE MONTRÉAL
affiliée à l'Université de Montréal

Ce mémoire intitulé :

A lightweight canonicalization engine in C

présenté par **William ZHANG**
en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
a été dûment accepté par le jury d'examen constitué de :

Danielle DUBOIS, présidente
Sébastien VIGOT, membre et directeur de recherche
Marie COUTURE, membre et codirectrice de recherche
Jean TREMBLAY, membre
Joseph BROWN, membre externe

DEDICATION

To all the past, present and future CVXPY contributors

ACKNOWLEDGEMENTS

Texte / Text.

RÉSUMÉ

Le résumé est un bref exposé du sujet traité, des objectifs visés, des hypothèses émises, des méthodes expérimentales utilisées et de l'analyse des résultats obtenus. On y présente également les principales conclusions de la recherche ainsi que ses applications éventuelles. En général, un résumé ne dépasse pas trois pages.

Le résumé doit donner une idée exacte du contenu du mémoire ou de la thèse. Ce ne peut pas être une simple énumération des parties du manuscrit. Le but est de présenter de façon précise et concise la nature, l'envergure de la recherche, les sujets traités, les questions de recherche ou les hypothèses soulevées, les méthodes utilisées, les principaux résultats ainsi que les conclusions retenues. Un résumé ne doit jamais comporter de références ou de figures.

ABSTRACT

Written in English, the abstract is a brief summary similar to the previous section (Résumé). However, this section is not a word for word translation of the abstract in French.

The abstract is a brief statement of the subject matter, objectives, research questions or hypotheses, experimental methods and analysis of results. It also presents the main research conclusions and their possible applications. In general, an abstract should not exceed three pages.

The abstract should provide an exact idea of the thesis or dissertation's contents and it cannot be a simple enumeration of the manuscript's parts. The goal is to precisely and concisely present the nature and scope of the research. An abstract should never include references or figures. If the thesis or the dissertation is in English, the résumé (French-language abstract) should come first followed by the abstract.

TABLE OF CONTENTS

DEDICATION iii

ACKNOWLEDGEMENTS iv

RÉSUMÉ v

ABSTRACT vi

LIST OF TABLES ix

LIST OF FIGURES x

LIST OF SYMBOLS AND ACRONYMS xi

CHAPTER 1 INTRODUCTION 1

CHAPTER 2 LITERATURE REVIEW 4
2.1 Canonicalization 6
2.2 Matrix stuffing 10
2.3 Coefficient extraction 10
2.4 Computing derivatives 12
2.5 Sparse matrices 15

CHAPTER 3 A lightweight canonicalization engine 18
3.1 Fundamental data structures 18
3.2 Atom implementations 19
3.3 Problem class and Parameter updates 25
3.4 CVXPY interface 25

CHAPTER 4 Benchmark results 26
4.1 Benchmark Setup 26
4.1.1 Example 26
4.1.2 Runtime and Memory Usage 26
4.2 Results on Standard Benchmarks 26
4.3 Results on Stress Test Problems 26
4.3.1 Least Squares Problems 26

CHAPTER 5 CONCLUSION 27

5.1 Synthèse des travaux / Summary of Works 27
5.2 Limitations de la solution proposée / Limitations 27
5.3 Améliorations futures / Future Research 27

REFERENCES 28

LIST OF TABLES

LIST OF FIGURES

Figure 2.1 Users write expressions in Python-code that is immediately parsed into an AST, shown here. DPP verification is then run on the AST to ensure canonicalization will be possible on the problem and to verify convexity. Variables are labeled with a circle, and parameters with a square. Leaf nodes that refer to the same parameter or variable are deduplicated in practice (not shown here). 9

Figure 2.2 After the application of reductions, the constraints and the objective are affine functions of the variables. As with the AST representation, duplicate leaves are deduplicated in practice, which is not shown here. 10

Figure 2.3 Matrix stuffing generates a 3D tensor. To depict these 3D tensors here, we leave the parameter values symbolic. Contrast this Figure with Fig. 2.2, which shows the operations being applied at each node. 11

Figure 2.4 Parameter map Q of the augmented objective $[c;\, d]$ for the running example. The first n rows index the entries of the gradient c (one per post-canonicalization variable: t_1, t_2, and the n-block x); the final row indexes the constant offset d (zero here, since the canonicalized objective $t_1 + \mu t_2$ has no offset). Columns index the global augmented parameter vector $\tilde{\theta} = (g,\, \mu,\, 1)$. The g columns are zero because the objective does not depend on g, but they are part of $\tilde{\theta}$ globally and the analogous map for the constraints is non-zero in those columns. Multiplying Q by $\tilde{\theta}$ recovers $[c;\, d] = [1,\, \mu,\, 0,\, 0]^T$; the leading three entries match the c vector shown in Fig. 2.3. 12

Figure 2.5 Parameter map for $[A\, b]$ in the running example. The 3D tensor R has one 2D slice per entry of $\tilde{\theta} = (g_1, \ldots, g_m, \mu, 1)$, and $[A\, b] = \sum_{i=1}^{m+2} \tilde{\theta}_i\, \mathcal{R}[:, :, i]$. Block-row order matches Fig. 2.3(b): $x \in \mathbf{R}^n_+$; t_1 row of SOC_1; $Fx - g$ block of SOC_1; t_2 row of SOC_2; x block of SOC_2. Block-columns are the canonical-form variables $(t_1,\, t_2,\, x)$ followed by the b column. Summing $g_i \mathcal{R}[:, :, g_i]$ over i regenerates the $-g$ entry in b shown at the root of Fig. 2.3(b); the μ slice contributes nothing; the constant slice contributes everything else. 13

LIST OF SYMBOLS AND ACRONYMS

AD	Automatic Differentiation
AML	Algebraic Modeling Language
ASA	Affine-Solver-Affine
CSC	Compressed Sparse Column
CSR	Compressed Sparse Row
DAG	Directed Acyclic Graph
DCP	Disciplined Convex Programming
DPP	Disciplined Parameterized Programming
LP	Linear Program
MOI	MathOptInterface
MPS	Mathematical Programming System
NLP	Nonlinear Program
QP	Quadratic Program
SOCP	Second-Order Cone Program
SDP	Semidefinite Program

CHAPTER 1 INTRODUCTION

Domain specific modeling languages for convex optimization such as CVXPY [1] and YALMIP [2] allow users to formulate optimization problems in a natural mathematical syntax. Through a process called *canonicalization*, these high-level problem descriptions are converted to a *standard conic form* used by many general-purpose convex solvers

$$\begin{array}{ll} \text{minimize} & c^T x \\ \text{subject to} & Ax + s = b \\ & s \in K, \end{array} \tag{1.1}$$

where $x \in \mathbf{R}^n$ and $s \in \mathbf{R}^m$ are the decision variables, and the *problem data* are the matrix $A \in \mathbf{R}^{m \times n}$, the vectors $b \in \mathbf{R}^m$ and $c \in \mathbf{R}^n$, and the non-empty closed convex cone $K \subseteq \mathbf{R}^m$. Almost all of optimization practice can be divided into two groups, *modeling* and *solving*. The modeler's role is to formulate problems based on real-world business or engineering applications. The solver's role is to find an optimal solution given the data of a problem instance. Modeling languages serve as the communication bridge between these two groups. When improvements in solvers are made, modeling languages must adapt to allow users to benefit from them. Conversely, it is through natural formulations that modelers can express their requirements and help motivate solver improvements.

While the time taken to solve an optimization problem is often dominated by the solver, the time taken to *canonicalize* (or *compile*) a problem can also be significant, especially for large problems. In the case of CVXPY, the *compilation time* largely lies in *matrix stuffing*, the final step of canonicalization where symbolic expressions trees are converted to explicit numerical matrices. Other modeling languages take different approaches to form the problem data, we review these *coefficient extraction* approaches in §2.3.

Atoms. Modeling languages for convex optimization are built on top of a library of functions which can be used to describe problems. These functions are known as *atoms* in CVXPY since they are the fundamental building blocks of the modeling language [3]. Atoms have known attributes such as curvature (affine, convex, concave), monotonicity (increasing, decreasing, non-monotone), and sign (positive, negative, unknown), which are used during canonicalization (we show an example in §2.1).

Parameters. The data inputs in a problem instance can either be fixed or varying. In CVXPY, varying data are referred to as *parameters* and fixed data are referred to as *constants*. Parameters are useful for specifying a *family* of problems where the structure is fixed [4]. During canonicalization, CVXPY will form a *parameter map* which maps each parameter to the indices of the problem data that depend on it. On subsequent solves, CVXPY will only need to update the parameter values and apply the parameter map to get an instance of problem data. We describe the parameter map and how to compute it in §2.2.

Expression trees. An expression is recursively defined as an atom evaluated at a *subexpression*. The subexpression can either be a variable, a parameter, or another expression. Similarly to atoms, expressions can also be classified by curvature, monotonicity, and sign which are determined by simple *composition rules* [5]. An expression tree whose atoms are all affine is known as an *affine expression tree*.

Matrix stuffing. Although the original user specified problem formulation may contain nonlinearities, after conversion to standard form, the *canonicalized problem* will only contain affine expressions. The convex cone $\mathcal{K}$ (typically a cartesian product of *standard cones*) is already computed and known. The remaining problem data are the *constraint matrix* A and the vectors b and c. The process to compute b is relatively straightforward, for example, one can *evaluate* the affine expression tree at the origin. The process to obtain c is a special case of the process to obtain A. For these reasons, we will often refer to matrix stuffing as the explicit task of forming the A matrix. We review current CVXPY matrix stuffing approaches in §2.2.

Contributions. We make the following contributions:

1. We propose an alternative method for matrix stuffing that doesn't rely on a parameter map. Instead, we treat matrix stuffing as a special case of sparse Jacobian evaluation which naturally separates the symbolic structure of the Jacobian from its numerical values.

2. We use the affine atoms from the `SparseDiffengine` package and its Python bindings to implement a backend for CVXPY.

3. We benchmark our method against existing approaches and make improvements to CVXPY's codebase by removing a deprecated backend.

All code is open source and available at `https://github.com/cvxpy/cvxpy`.

Thesis outline. The rest of this thesis begin with a review of the literature on modeling languages, canonicalization, and matrix stuffing in §2. In §3 we describe a set of relevant affine atoms and their implementation in a sparse differentiation package. In §4 we present benchmark results comparing our method to existing approaches. Finally, in §5 we summarize our contributions and discuss future work.

CHAPTER 2 LITERATURE REVIEW

Solving an optimization problem on a computer requires translating the mathematical formulation into a data format that solvers can process. For linear programs, this means constructing the constraint matrix A, the right-hand side vector b, and the objective coefficients c that define the *standard form*,

$$\begin{aligned} \text{minimize} \quad & c^T x \\ \text{subject to} \quad & Ax \leq b, \\ & x \geq 0. \end{aligned} \tag{2.1}$$

Manual encoding In the early days of linear programming, analysts manually computed and encoded each coefficient onto punch cards. This process was tedious, error-prone and became impractical as linear programming found more and more applications in large-scale industrial problems [6].

MPS format IBM's Mathematical Programming System (MPS) file format, developed in the 1960s, established a standardized file format for representing linear programs [7]. The MPS format uses fixed-width fields designed for 80-column punch cards, with sections for rows, columns, and coefficients. William Orchard-Hays [8] provides a comprehensive history of these early mathematical programming systems, tracing their development from 1952 at RAND Corporation through their widespread adoption on mainframe computers in the late 1960s.

Matrix generators To automate the creation of MPS files, *matrix generators* emerged in the late 1960s; they are procedural programs that compute the problem coefficients through explicit loops and output the results in MPS format [9]. Commercial systems such as `MaGen` [10], `GAMMA` [11], and `DATAFORM` [12] were among the first matrix generators which were used in various domains.

A modern approach Modelers naturally think in terms of sets, indices, and algebraic expressions, not in terms of individual matrix entries. Matrix generators simplified the encoding process but still required modelers to translate their mathematical formulations into procedural code that explicitly constructs the coefficient matrices. This suggests that the gap between the modeler's representation of the problem and the solver's required format

hadn't been resolved yet [6].

Algebraic modeling languages In the late 1970s, *algebraic modeling languages* emerged as formal languages designed to accept problem descriptions that closely mirror standard mathematical notation [13]. By working directly with the modeler's representation, these languages simplify verification (the model can be inspected as written), modification (changing the problem does not require rewriting procedural code), and communication (the formulation serves as its own documentation). In the late 1980s and early 1990s, commercial algebraic modeling systems such as GAMS [14], AIMMS [15] and AMPL [16] became widely adopted in both academia and industry (and they are still widely used today).

Conic standard form In the 2000s, advances in convex optimization made it practical to solve a much broader class of problems efficiently. Solvers such as MOSEK [17, 18] adopted *conic optimization* as their interface—a structured subclass of convex optimization with the standard form [**?**]:

$$\begin{array}{ll} \text{minimize} & c^T x \\ \text{subject to} & b - Ax \in \mathcal{K}, \end{array} \tag{2.2}$$

where $\mathcal{K}$ is a closed convex cone. This formulation generalizes linear programming (when $\mathcal{K}$ is the nonnegative orthant) to include second-order cone programs (SOCPs), semidefinite programs (SDPs) and many more.

Embedded modeling languages In the late 2000s and early 2010s, modeling languages emerged that are embedded in general-purpose programming languages (in contrast with standalone systems like AMPL and GAMS). Examples include YALMIP [2] in MATLAB, Pyomo [19] in Python, JuMP [20] in Julia, and CasADi [21] in C++.

Disciplined convex programming In addition, there are domain specific languages that implement *disciplined convex programming* (DCP), a set of composition rules that can automatically verify convexity [3, 5]. These include CVX [22] in MATLAB, CVXPY [1, 23] in Python, and Convex.jl [24] in Julia.

DCP composition rules. The expression $h(f_1(x), f_2(x), \ldots, f_k(x))$ is convex when h is convex and for each $i \in \{1, \ldots, k\}$ the following holds:

- h is non-decreasing in argument i and f_i is convex, or
- h is non-increasing in argument i and f_i is concave, or

- f_i is affine.

The same results hold for concave functions by swapping convex and concave. Affine expressions require h and all of its arguments to be affine [**?**].

Structure of the chapter

2.1 Canonicalization

In this section we cover the canonicalization process in CVXPY.

CVXPY. CVXPY is a domain-specific language for convex optimization embedded in Python, that was originally developed by Steven Diamond and Stephen Boyd at Stanford University [1]. Akshay Agrawal later redesigned the rewriting system in CVXPY [23], which enabled support for other disciplined languages [25–27], and the project has since grown through contributions from many others [**?**].

Standard functional form. CVXPY problems are specified in the standard "functional" form for convex optimization,

$$\begin{array}{ll} \text{minimize} & f_0(x;\theta) \\ \text{subject to} & f_i(x;\theta) \leq 0,\ i = 1,\ldots,m_1, \\ & g_i(x;\theta) = 0,\ i = 1,\ldots,m_2, \end{array} \tag{2.3}$$

with variable $x \in \mathbf{R}^n$ and parameter vector $\theta \in \mathbf{R}^p$ [**?**]. When θ is fixed, (2.3) is a problem instance, otherwise it defines a problem family. Each function in (2.3) is internally represented as an *expression tree* (or *abstract syntax tree*, AST): each parent node is an atomic function (an *atom*), its children are the atom's arguments, and leaf nodes are variables, parameters, or constants. Convexity is verified by applying the DCP composition rule (2) to every node of each AST; expressions like $\sqrt{x^2}$ are rejected while the equivalent $|x|$ is accepted.

Signed DCP. In addition to per-argument monotonicities, CVXPY tracks sign information to extend the set of valid compositions. Without signed DCP, $f(g(x)) = (|x|)^2$ is invalid since $f(t) = t^2$ is not monotone on $\mathbf{R}$. With signed DCP, it is valid because f is non-decreasing on $\mathbf{R}_+$ and the image of g lies in $\mathbf{R}_+$.

Solver standard form. Canonicalization produces a problem in a *solver standard form* expected by the solver. Common families include linear programs, quadratic programs, cone programs of the form (2.2), and *conic quadratic programs* which combine a quadratic objective with cone constraints. The conic quadratic form is the most general of these, but is not a major generalization of the cone form: any conic quadratic program can be reduced to an equivalent cone program by linearizing the quadratic objective via the second-order cone, at the cost of one extra variable and one extra constraint. We therefore focus on canonicalization to the cone form throughout this thesis.

Parameterized conic programming. A *parameterized cone program* extends (2.2) by letting the data depend on a parameter vector $\theta \in \mathbf{R}^p$:

$$\begin{array}{ll} \text{minimize} & c(\theta)^T x + d(\theta) \\ \text{subject to} & b(\theta) - A(\theta)x \in \mathcal{K}. \end{array} \tag{2.4}$$

When the maps c, d, A, b are *affine in* θ, this is the *affine-solver-affine* (ASA) form, and the original solution is recovered by an affine map from the solution of the canonicalized problem. *Disciplined parameterized programming* (DPP) is a grammar built on top of DCP (2) that disallows products of parameter-dependent expressions, guaranteeing reducibility to ASA form.

For a DPP-compliant problem, the canonicalization map admits a compact representation: there exist sparse tensors $Q \in \mathbf{R}^{(n+1)\times(p+1)}$ and $\mathcal{R} \in \mathbf{R}^{m\times(n+1)\times(p+1)}$ such that, with $\tilde{\theta} = (\theta, 1) \in \mathbf{R}^{p+1}$,

$$\begin{bmatrix} c \\ d \end{bmatrix} = Q \cdot \tilde{\theta}, \qquad \text{and} \qquad [A \quad b] = \sum_{i=1}^{p+1} \mathcal{R}[:, :, i] \cdot \tilde{\theta}_i, \tag{2.5}$$

where d is the parameter-dependent constant offset of the objective. See [27] for details.

Stages of canonicalization. Canonicalization converts a verified DCP/DPP model into a cone program via a sequence of reductions [**?**, 23]. The front end verifies convexity by parsing the user specified expression tree. Then, reductions—*e.g.*, negating the objective of maximization problems and turning variable attributes into explicit constraints—are applied. The last symbolic reduction is to expand nonlinear atoms to their epigraph form, resulting in an affine expression tree. Subsequently, a numerical reduction reduces the affine expression tree into the sparse tensors Q and $\mathcal{R}$ defined above. Finally, solver-specific transformations are applied to match the form required by the solvers' interfaces. At each reduction, the inverse transformations are also recorded, allowing us to map the solution of the standard

form problem back to the user specified problem.

Example. As an illustrative example, consider the following optimization problem

$$\begin{array}{ll} \text{minimize} & \|Fx - g\|_2 + \mu\|x\|_2 \\ \text{subject to} & x \succeq 0 \end{array} \tag{2.6}$$

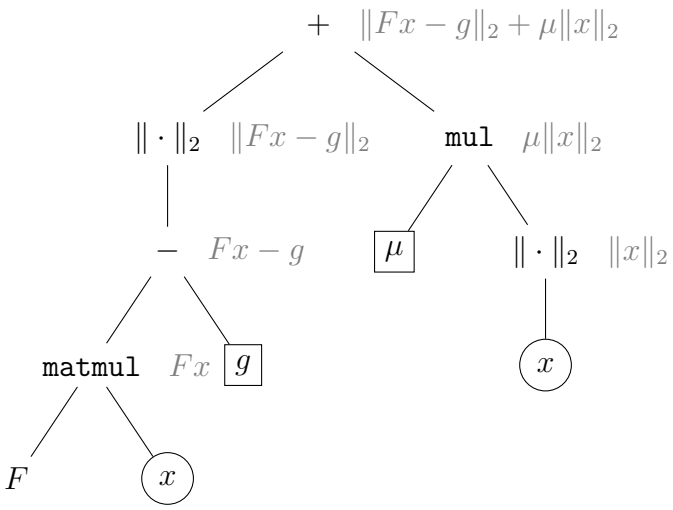
with variable $x \in \mathbf{R}^n$, fixed problem data $F \in \mathbf{R}^{m\times n}$, and parameters $g \in \mathbf{R}^m$ and $\mu \geq 0$. This problem formulation, given by the following CVXPY code, conforms to the DPP grammar:

```
import cvxpy as cp
import numpy as np
F = np.random.randn(m, n) # fixed problem data
x = cp.Variable(n)
g = cp.Parameter(m)
mu = cp.Parameter(nonneg=True)
objective = cp.Minimize(cp.norm(F @ x - g) + mu * cp.norm(x))
prob = cp.Problem(objective, [x >= 0])
```

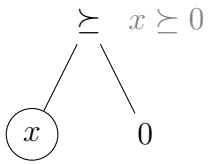
In the following we describe in greater detail the entire canonicalization procedure for problem (2.6). In the first phase, called *parsing*, the front end builds the AST of each function and verifies the DCP rules [23]; if a function does not conform, CVXPY raises an error. Figure 2.1 shows the AST for the running example.

Reductions. Following the parsing and DPP verification steps, reductions are applied to transform the problem into a cone program. In DPP, each atom has a *canonicalization* that replaces nonlinear functions with an epigraph variable and auxiliary constraints of the form $f(x, t, \theta) \in \mathcal{K}$ where t is the epigraph variable, f is affine in the variables (x, t) for fixed parameters θ, and $\mathcal{K}$ is a convex cone. One illustrative canonicalization replaces the $\|\cdot\|_2$ expressions in the objective with a new epigraph variable t_i and imposes the epigraph constraint in its conic form

$$(t_i, z_i) \in \mathcal{K}_{\text{soc}}. \tag{2.7}$$



(a) Abstract syntax tree of objective



(b) Abstract syntax tree of constraints

Figure 2.1 Users write expressions in Python-code that is immediately parsed into an AST, shown here. DPP verification is then run on the AST to ensure canonicalization will be possible on the problem and to verify convexity. Variables are labeled with a circle, and parameters with a square. Leaf nodes that refer to the same parameter or variable are deduplicated in practice (not shown here).

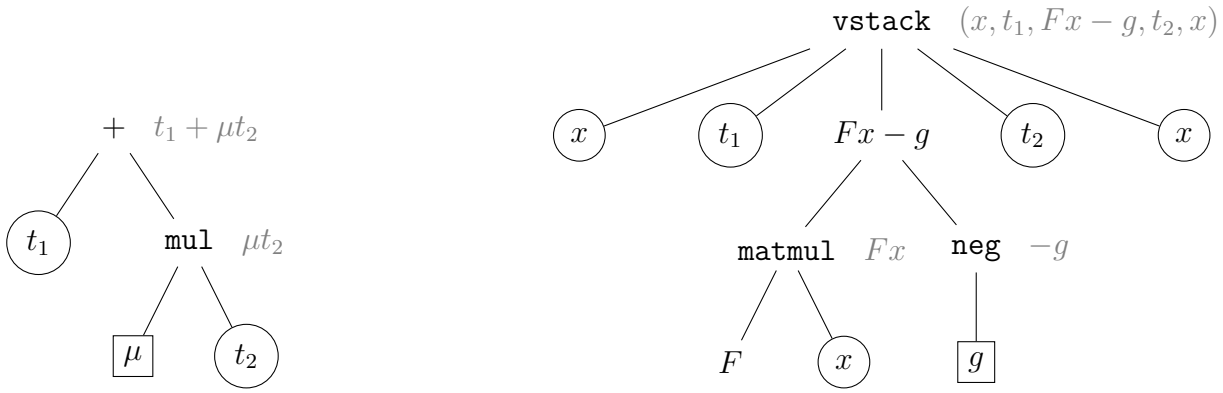
Expression tree. Following the reductions, the problem has been reduced to an equivalent parameterized convex cone program:

$$\begin{array}{ll} \text{minimize} & t_1 + \mu t_2 \\ \text{subject to} & x \in \mathbf{R}^n_+ \\ & (t_1, Fx - g) \in \mathcal{K}_{\text{soc},m+1} \\ & (t_2, x) \in \mathcal{K}_{\text{soc},n+1} \end{array} \tag{2.8}$$

We represent the constraints and the objective (which is affine after reductions) as expression trees, see Figure 2.2.

The final step is to materialize these expression trees as the sparse tensors Q and $\mathcal{R}$ defined above; this step, called *matrix stuffing*, is described in §2.2.

References.



(a) Expression tree of the objective

(b) Expression tree of the constraints

Figure 2.2 After the application of reductions, the constraints and the objective are affine functions of the variables. As with the AST representation, duplicate leaves are deduplicated in practice, which is not shown here.

2.2 Matrix stuffing

Matrix stuffing materializes the expression trees of the canonicalized problem (§2.1) as the sparse tensors Q and $\mathcal{R}$ defined in §2.1. This is done by the *canonicalization backend*, which applies a depth-first walk on each tree (Figure 2.3).

References.

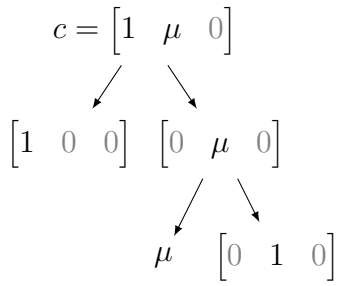
2.3 Coefficient extraction

`JuMP` and `MathOptInterface`. JuMP is an algebraic modeling language embedded in Julia, originally developed in 2012 by Iain Dunning, Joey Huchette, and Miles Lubin at the MIT Operations Research Center [20, 28]. Its abstraction layer, MathOptInterface (MOI) [29], rethinks how a modeling language communicates with solvers, moving away from the traditional approach of reducing every problem to a single standard form.

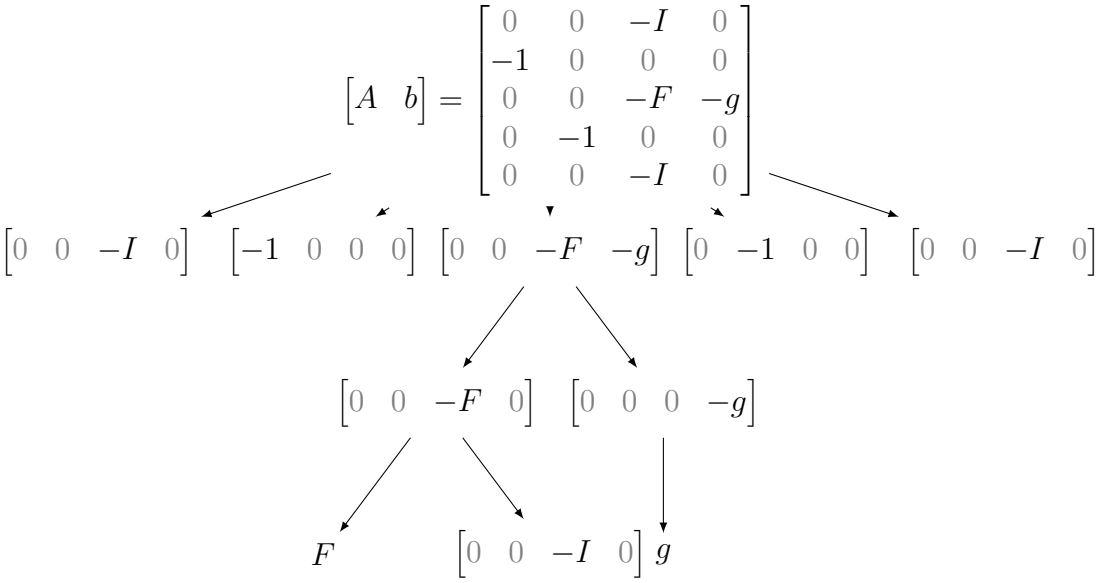
Function-set pairs

`CasADi`. [21]

`AMPL`. [16]



(a) Matrix stuffing of the objective



(b) Matrix Stuffing tree

Figure 2.3 Matrix stuffing generates a 3D tensor. To depict these 3D tensors here, we leave the parameter values symbolic. Contrast this Figure with Fig. 2.2, which shows the operations being applied at each node.

$$Q = \begin{array}{c} \\ t_1 \\ t_2 \\ x \\ d \end{array} \begin{array}{c} \begin{array}{ccc} g^T & \mu & 1 \end{array} \\ \begin{bmatrix} 0 \cdots 0 & 0 & 1 \\ 0 \cdots 0 & 1 & 0 \\ 0 \cdots 0 & 0 & 0 \\ 0 \cdots 0 & 0 & 0 \end{bmatrix} \end{array} \qquad \begin{bmatrix} c \\ d \end{bmatrix} = Q\,\tilde{\theta} = \begin{bmatrix} 1 \\ \mu \\ 0 \\ 0 \end{bmatrix}$$

Figure 2.4 Parameter map Q of the augmented objective $[c;\, d]$ for the running example. The first n rows index the entries of the gradient c (one per post-canonicalization variable: t_1, t_2, and the n-block x); the final row indexes the constant offset d (zero here, since the canonicalized objective $t_1+\mu t_2$ has no offset). Columns index the global augmented parameter vector $\tilde{\theta} = (g,\, \mu,\, 1)$. The g columns are zero because the objective does not depend on g, but they are part of $\tilde{\theta}$ globally and the analogous map for the constraints is non-zero in those columns. Multiplying Q by $\tilde{\theta}$ recovers $[c;\, d] = [1,\, \mu,\, 0,\, 0]^T$; the leading three entries match the c vector shown in Fig. 2.3.

References.

2.4 Computing derivatives

Finding the rate of change or the *derivative* of a function at a particular point is a fundamental concept in calculus that was discovered by Leibniz and Newton in the late 1600s. Since then, although the underlying theory has practically remained the same, various methods have been developed to efficiently compute derivatives in different contexts and applications. More recently, the rise of machine learning and deep learning brought renewed interest in this age-old problem through a new application of derivatives in training *neural networks*, it is known as the *backward pass* or *backpropagation* [30, 31]. Some theorists will claim that backpropagation is little more than a clever application of the *chain rule* [32], but the large scale of modern workloads has led to the development of incredible software tools and implementation techniques which are interesting in their own right. This section provides an overview of these new tools and techniques, with a particular emphasis on *algorithmic differentiation* (AD) and its applications to computing sparse Jacobians [33].

Numerical differentiation. Other than computing derivatives by hand, the simplest method is to use *numerical differentiation*, which approximates the derivative of a function at a point by evaluating the function at nearby points using a *finite difference* formula similar to either

$$f'(x) \approx \frac{f(x+h) - f(x)}{h}, \tag{2.9}$$

$$\mathcal{R}[:,:,g_i] = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -e_i \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \qquad \mathcal{R}[:,:,\mu] = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

(a) Parameter slices. Each $\mathcal{R}[:,:,g_i]$ $(i = 1, \ldots, m)$ places a single -1 in the b column at the i-th row of the $Fx - g$ block (denoted $-e_i$). $\mathcal{R}[:,:,\mu] = 0$ because μ does not appear in any constraint.

$$\mathcal{R}[:,:,1] = \begin{bmatrix} 0 & 0 & -I & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & -F & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & -I & 0 \end{bmatrix}$$

(b) Constant slice. Holds the fixed structure: $-I$ blocks for the variable bound and SOC_2 rows, -1's for the epigraph variables, and $-F$ for the Fx block.

Figure 2.5 Parameter map for $[A\, b]$ in the running example. The 3D tensor R has one 2D slice per entry of $\tilde{\theta} = (g_1, \ldots, g_m, \mu, 1)$, and $[A\, b] = \sum_{i=1}^{m+2} \tilde{\theta}_i \, \mathcal{R}[:,:,i]$. Block-row order matches Fig. 2.3(b): $x \in \mathbf{R}^n_+$; t_1 row of SOC_1; $Fx - g$ block of SOC_1; t_2 row of SOC_2; x block of SOC_2. Block-columns are the canonical-form variables (t_1, t_2, x) followed by the b column. Summing $g_i \mathcal{R}[:,:,g_i]$ over i regenerates the $-g$ entry in b shown at the root of Fig. 2.3(b); the μ slice contributes nothing; the constant slice contributes everything else.

or

$$f'(x) \approx \frac{f(x+h) - f(x-h)}{2h}, \tag{2.10}$$

where h is a small step size. Although this approach is straightforward to implement and can achieve reasonable accuracy, there are important downsides to consider. First, it requires a linear number of function evaluations with respect to the number of input variables, which is expensive for high-dimensional functions. Second, it can easily lead to *truncation errors* (when h is too large) or *rounding errors* (when h is too small) in the computed derivative if the step size is not carefully chosen [34]. Despite these downsides, numerical differentiation is still widely used in practice, especially for verifying the correctness of derivative computations (also known as a *derivative checker*) [35].

Symbolic differentiation. Mathematical expressions can be manipulated automatically using simple derivative rules of calculus, such as the product and sum rule. This *symbolic* approach to differentiation is used in computer algebra systems such as Mathematica [36] and Maple [37] as well as the SymPy library in Python [38]. Unlike its numerical counterpart,

symbolic differentiation tools compute derivatives as closed-form expressions, which can be simplified and evaluated at any arbitrary point. These closed-form expressions are particularly useful for deriving analytic solutions of optimization problems, and can be further used in mathematical proofs and analyses.

However, a naive implementation of symbolic differentiation can easily lead to exponentially large symbolic expressions, which is impractical and inefficient for many problems [33]. Interestingly, when intermediate expressions are stored as *common subexpressions*, and the expression is represented with a *directed acyclic graph* (DAG) instead of a tree, symbolic differentiation performs the same set of operations as *reverse mode differentiation* [39].

Algorithmic differentiation. There is a lot of confusion on the terminology surrounding the various methods for computing derivatives; *automatic differentiation* and *computational differentiation* are often used to describe the field, but we will instead refer to it as *algorithmic differentiation* since it emphasizes the algorithms behind the different implementations rather than a particular method (it is also the convention in the seminal book by Griewank and Walther [33]). Fundamentally, AD is a non-standard interpretation of a computer program: it augments the *evaluation trace* of a function (the sequence of floating point values calculated during the execution) with calculations of its derivatives [40]. This trace can be represented as a *computational graph*, a DAG in which each node corresponds to an elementary operation with known derivatives. Combining the derivatives of these operations throughout the graph using the chain rule gives the derivative of the overall function [33]. The computational graph is the central data structure of AD; the various *modes* of AD correspond to different ways elementary derivatives are accumulated on this graph.

Forward mode AD. The method of propagating tangents forward through the computational graph is known as *forward mode* AD. A single forward pass simultaneously computes the value and derivative of every intermediate variable with respect to a single input variable, giving the sensitivity of all outputs to that input. The cost of a forward pass is a small constant factor of the cost of evaluating the original function, since each elementary operation is augmented with at most a few additional arithmetic operations for the derivatives. However, one downside of forward mode AD is that the number of forward passes scales linearly with the number of input variables, which is impractical for certain functions (which have many input variables and very few output variables).

Reverse mode AD. In contrast to forward mode, *reverse mode* AD propagates adjoints backward through the computational graph. It first requires a forward pass to compute and

store the value of every intermediate variable, and it then performs a reverse pass to compute the sensitivities of a single output variable with respect to all input variables. The cost of a reverse pass is also a small constant factor of the cost of evaluating the original function, but the number of passes required now scales linearly with the number of output variables rather than the number of input variables. This makes reverse mode AD particularly efficient for computing gradients of scalar-valued functions, making it the default method for modern machine learning libraries such as Jax [41], Pytorch [42], and Tensorflow [43].

Advantages of AD. [44].

Jacobian and Hessian computation.

Sparse Jacobian computation. graph coloring, cross-country elimination, vertex elimination, jacobian accumulation, etc. maybe mention graph coloring here?

[45] presents a code generator for operations on sparse data.

[46] presents a similar method for computing sparse Hessians with graph coloring. [47] is a survey of graph coloring methods for computing sparse Jacobians and Hessians.

Differentiation packages. mention ADOL-C and CppAD for C++. mention AMPL and CasADi for optimization. and also mention that they were covered above in the coeff extraction section.

References. The book by Griewank and Walther [33] is a comprehensive reference on the theory and practice of automatic differentiation. The book by Naumann [44] is another great reference that focuses on the implementation of AD in software, with a particular emphasis on the use of operator overloading and source code transformation techniques. [40] is an excellent and relatively up-to-date survey of the various methods for automatic differentiation in machine learning.

2.5 Sparse matrices

Matrices that contain many number of zero entries are known as *sparse matrices*. The simplest sparse matrix data structure is the *coordinate* or *triplet* format. It consists of two integer index arrays `rows` and `cols` and a floating point value array `vals` of length equal to the number of *nonzero entries* in the matrix.

For example, the matrix

$$\begin{bmatrix} 0 & 0 & 3.2 & 0 \\ 4.1 & 1.8 & 0 & 2.2 \\ 0 & 2.8 & 0 & 3.4 \end{bmatrix} \tag{2.11}$$

can be represented in coordinate format as

```
int    rows[] = {0,  1,   1,   1,   2,   2};
int    cols[] = {2,  0,   1,   3,   1,   3};
double vals[] = {3.2, 4.1, 1.8, 2.2, 2.8, 3.4};
```

where we use zero-based indexing and array declaration notation from the C programming language [48]. The coordinate format is the easiest to understand and form, but it is not effective for most operations on sparse matrices; the *compressed sparse row* or CSR format is commonly used instead.

Compressed sparse row format. The CSR format differs from the coordinate format in that it replaces the `rows` array with a row pointer array `idxptr` of length equal to the number of rows plus one. The row pointer array indicates the starting index of each row in the `cols` arrays. For example, the matrix in (2.11) can be represented in CSR format as

```
int  idxptr[] = {0,  1,              4,        6};
int   cols[] = {2,  0,   1,   3,   1,   3};
double vals[] = {3.2, 4.1, 1.8, 2.2, 2.8, 3.4};
```

where we deliberately format `idxptr` to align with the start of each row in the `cols` arrays. The benefits of the CSR format are more than a reduced size of the `idxptr` array compared to the `rows` array. In the CSR format, accessing a row takes constant time and is very *cache-friendly* since the nonzero entries are stored contiguously in memory. For this reason, the CSR format is the default for *iterative methods* to solve sparse linear systems [49].

Compressed sparse column format. However, one downside of the CSR format is that accessing a column is much more difficult; one would need to look at every row to find the nonzero entries in a column. An alternative format is the *compressed sparse column* or CSC format, which swaps the role of the `idxptr` array to refer to column indices instead. In contrast to iterative methods, the CSC format is the preferred format for *direct methods* to solve sparse linear systems [50].

Transpose.

Which format to use?

Other formats. The coordinate, CSR and CSC formats are the most widely used sparse matrix formats, but many others exist for different specialized use cases. The scientific Python sparse library `scipy.sparse` [51] allows users to form other types of sparse matrices such as the list of lists (LIL) matrix, the dictionary of keys (DOK) matrix, and the block-sparse row (BSR) matrix.

Sparse matrix struct in C. A CSR matrix struct in C looks like this.

```
typedef struct CSR_Matrix {
    int *p;      /* row pointers, size m+1 */
    int *i;      /* column indices, size nnz */
    double *x;   /* values, size nnz       */
    int m, n, nnz;
} CSR_Matrix;
```

References. Although the work in this thesis does not necessarily revolve around solving sparse linear systems, a lot of techniques for manipulating sparse matrices we use were inspired by the literature. In particular, the book by Davis [50] was used extensively throughout the development of our methods. The book by Duff [52] is another great reference to learn more about the history and developement of sparse matrix algorithms. Another reference is the review article by the "gang of four" (Gill, Murray, Saunders and Wright) from the Systems Optimization Laboratory at Stanford University [53] which provides a high-level overview of various applications of sparse matrix methods in optimization.

CHAPTER 3 A lightweight canonicalization engine

In this chapter we present a minimal implementation of a subset of affine atoms in the `SparseDiffEngine` package, and show how they can be used to compute the sparse constant Jacobian of a set of affine expressions. At this point, we are given a *canonicalized* problem in standard form, after the canonicalization process described in §2.1. The content of this chapter has many similarities to §2.2, since both serve the same purpose of forming the problem data. The structure of the chapter is as follows: in §3.1 we describe basic data structures in the engine which will be used throughout the atom implementations described in §3.2, and finally, in §3.4 we present a high level problem class interface which can easily be used in CVXPY. We also recommend reading §2.5 before reading this chapter, since it covers the basics of sparse matrices and their data structures.

3.1 Fundamental data structures

The main data structure in the engine is the expression struct.

```
typedef struct expr {
    int   d1, d2, size; /* shape (d1 x d2) and size = d1*d2 */
    int   n_vars;       /* total number of variables     */
    int   var_id;       /* offset into u, or NOT_A_VARIABLE */

    struct expr *left;  /* left child (NULL for leaves)  */
    struct expr *right; /* right child (NULL for leaves) */

    double    *value;   /* forward-pass output, size entries */
    CSR_Matrix *jacobian; /* size x n_vars sparse Jacobian */

    forward_fn     forward;
    jacobian_init_fn jacobian_init_impl;
    eval_jacobian_fn eval_jacobian;
} expr;
```

every expression node has a shape (d1 x d2) and a size (d1*d2), and also two pointers to its left and right children. It will also have a pointer to its value, which is the output of the forward pass, and a pointer to its Jacobian, which is a CSR matrix. Every expression must also implement three functions: a forward function, a Jacobian initialization function, and a

Jacobian evaluation function.

Leaf nodes. The two leaf nodes are variables and parameters. They are the fundamental building blocks of the expression tree.

3.2 Atom implementations

We will now show the implementation of atoms in the engine. The semantics of an atom is as follows, it takes one or two expressions as input, and produces an output expression corresponding to the result of the operation. Every atom implementation below will first show the basic CVXPY code, and then show how it is implemented in the engine.

Helpers and definitions. We first import the necessary libraries and define basic variables and constants which will be used throughout the implementation.

```
import cvxpy as cp
import numpy as np
import scipy.sparse as sp

x = cp.Variable(n1)
y = cp.Variable(n2)
a = cp.Parameter(n1)
b = cp.Parameter(n2)
```

Addition. The addition atom (also commonly referred to as `add`) takes two expressions and sums them together elementwise. The CVXPY code is as follows (note that Python's operator overloading for `+` will automatically form an `add` atom).

```
add_expr = cp.add(x, y)
```

In the engine the implementation is as follows.

```
static void forward(expr *node, const double *u)
{   /* forward pass for children */
    node->left->forward(node->left, u);
    node->right->forward(node->right, u);
    for (int i = 0; i < node->size; i++)
    {
        node->value[i] = node->left->value[i] + node->right->value[i];
```

```
    }
}

static void jacobian_init_impl(expr *node)
{
    /* initialize children's jacobians */
    jacobian_init(node->left);
    jacobian_init(node->right);

    /* we never have to store more than the sum of children's nnz */
    int nnz_max = node->left->jacobian->nnz + node->right->jacobian->nnz;
    node->jacobian = new_csr_matrix(node->size, node->n_vars, nnz_max);

    /* fill sparsity pattern */
    sum_csr_alloc();
}

static void eval_jacobian(expr *node)
{
    /* evaluate children's jacobians */
    node->left->eval_jacobian(node->left);
    node->right->eval_jacobian(node->right);

    /* sum children's jacobians */
    sum_csr_fill_values();
}
```

Negation. The negation atom takes a single expression and negates it elementwise.

Transpose. The transpose atom takes a single expression and transposes it. In CVXPY the code is as follows (it can also be written as `x.T`).

```
transpose_expr = cp.transpose(x)
```

```
// Forward pass for transpose atom
static void forward(expr *node, const double *u)
{
    /* forward pass for child */
    node->left->forward(node->left, u);
```

```
    /* local forward pass */
    int d1 = node->d1;
    int d2 = node->d2;
    double *X = node->left->value;
    double *XT = node->value;
    for (int i = 0; i < d1; ++i)
    {
        for (int j = 0; j < d2; ++j)
        {
            XT[j * d1 + i] = X[i * d2 + j];
        }
    }
}
```

explain the Jacobian init and eval for transpose, which is just a permutation of the Jacobian entries.

Horizontal stack. The horizontal stack atom takes two expressions and concatenates them horizontally. In CVXPY the code is as follows, and just like `NumPy`, it requires the input expressions to have the same number of rows.

```
hstack_expr = cp.hstack([x, y])
```

```
static void forward(expr *node, const double *u)
{
    hstack_expr *hnode = (hstack_expr *) node;

    /* children's forward passes */
    for (int i = 0; i < hnode->n_args; i++)
    {
        hnode->args[i]->forward(hnode->args[i], u);
    }

    /* concatenate values horizontally */
    int offset = 0;
    for (int i = 0; i < hnode->n_args; i++)
    {
        expr *child = hnode->args[i];
```

```
        memcpy(node->value + offset, child->value, child->size * sizeof(double
            ));
        offset += child->size;
    }
}

static void eval_jacobian(expr *node)
{
    hstack_expr *hnode = (hstack_expr *) node;
    CSR_Matrix *A = node->jacobian;
    A->nnz = 0;

    for (int i = 0; i < hnode->n_args; i++)
    {
        expr *child = hnode->args[i];
        child->eval_jacobian(child);

        /* copy values */
        memcpy(A->x + A->nnz, child->jacobian->x,
               child->jacobian->nnz * sizeof(double));
        A->nnz += child->jacobian->nnz;
    }
}
```

Vertical stack. The vertical stack atom takes two expressions and concatenates them vertically. In CVXPY the code is as follows, and just like `NumPy`, it requires the input expressions to have the same number of columns.

```
vstack_expr = cp.vstack([x, y])
```

In the engine the implementation uses the transpose and horizontal stack atoms, since we can write `vstack([x, y]) = transpose(hstack(transpose(x), transpose(y)))`.

More generally, we have the following equivalence:

```
vstack([A, B]) == transpose(hstack([transpose(A), transpose(B)]))
```

```
index_expr = x[0:3]
```

```
static void forward(expr *node, const double *u)
```

```
{
    expr *x = node->left;
    index_expr *idx = (index_expr *) node;

    /* child's forward pass */
    x->forward(x, u);

    /* gather selected elements */
    for (int i = 0; i < idx->n_idxs; i++)
    {
        node->value[i] = x->value[idx->indices[i]];
    }
}
```

mention that the Jacobian is a subset of the child's Jacobian, and we can fill it in by copying the relevant rows.

```
sum_expr = cp.sum(x)

static void forward(expr *node, const double *u)
{
    /* child's forward pass */
    x->forward(x, u);
    sum = 0.0;
    end = x->d1 * x->d2;
    /* sum all elements */
    for (i = 0; i < end; i++)
    {
        sum += x->value[i];
    }
    node->value[0] = sum;
}
```

```
elemwise_mult_expr = cp.multiply(x, y)

static void forward(expr *node, const double *u)
```

```
{
    expr *child = node->left;
    vector_mult_expr *vnode = (vector_mult_expr *) node;

    /* call forward for param_source expr tree (this extra logic is needed
       in case the parameter is a broadcast or promote node which needs to
          refresh
       its values) */
    if (vnode->base.needs_parameter_refresh)
    {
        vnode->param_source->forward(vnode->param_source, NULL);
        vnode->base.needs_parameter_refresh = false;
    }

    const double *a = vnode->param_source->value;

    /* child's forward pass */
    child->forward(child, u);

    /* local forward pass: elementwise multiply by a */
    for (int i = 0; i < node->size; i++)
    {
        node->value[i] = a[i] * child->value[i];
    }
}

static void jacobian_init_impl(expr *node)
{
    expr *x = node->left;

    /* initialize child jacobian */
    jacobian_init(x);

    /* same sparsity as child */
    node->jacobian = new_csr_copy_sparsity(x->jacobian);
}

static void eval_jacobian(expr *node)
```

```
{
    expr *x = node->left;
    const double *a = ((vector_mult_expr *) node)->param_source->value;

    /* evaluate x */
    x->eval_jacobian(x);

    /* row-wise scale child's jacobian */
    for (int i = 0; i < node->size; i++)
    {
        for (int j = x->jacobian->p[i]; j < x->jacobian->p[i + 1]; j++)
        {
            node->jacobian->x[j] = a[i] * x->jacobian->x[j];
        }
    }
}
```

Matrix multiplication.

3.3 Problem class and Parameter updates

3.4 CVXPY interface

CHAPTER 4 Benchmark results

4.1 Benchmark Setup

4.1.1 Example

4.1.2 Runtime and Memory Usage

4.2 Results on Standard Benchmarks

4.3 Results on Stress Test Problems

4.3.1 Least Squares Problems

CHAPTER 5 CONCLUSION

Texte / Text.

5.1 Synthèse des travaux / Summary of Works

Texte / Text.

5.2 Limitations de la solution proposée / Limitations

5.3 Améliorations futures / Future Research

Texte / Text.

REFERENCES

[1] S. Diamond and S. Boyd, "CVXPY: A Python-embedded modeling language for convex optimization," *Journal of Machine Learning Research*, vol. 17, no. 83, pp. 1–5, 2016.

[2] J. Löfberg, "YALMIP: A toolbox for modeling and optimization in MATLAB," in *2004 IEEE international conference on robotics and automation (IEEE Cat. No. 04CH37508)*. IEEE, 2004, pp. 284–289.

[3] M. Grant, "Disciplined Convex Programming," Ph.D. dissertation, Stanford University, Dec. 2004, ph.D. dissertation. [Online]. Available: https://web.stanford.edu/~boyd/papers/pdf/mcg_thesis.pdf

[4] M. Schaller, G. Banjac, S. Diamond, A. Agrawal, B. Stellato, and S. Boyd, "Embedded code generation with cvxpy," *IEEE Control Systems Letters*, vol. 6, pp. 2653–2658, 2022.

[5] M. Grant, S. Boyd, and Y. Ye, "Disciplined convex programming," in *Global Optimization: From Theory to Implementation*, L. Liberti and N. Maculan, Eds. New York: Springer, 2006, pp. 155–210.

[6] R. Fourer and M. J. Harrison, "A modern approach to computer systems for linear programming," 1978.

[7] *IBM Mathematical Programming System Extended/370 (MPSX/370) Program Reference Manual*, 2nd ed., IBM Corporation, 1976.

[8] W. Orchard-Hays, "History of mathematical programming systems," *Annals of the History of Computing*, vol. 6, no. 3, pp. 296–312, 1984.

[9] I. Marceau and T. W. Mason, "A matrix-generator system for linear programming problems," Center for Advanced Computation, University of Illinois at Urbana-Champaign, Urbana, IL, Tech. Rep. CAC Document No. 4, 1971. [Online]. Available: https://hdl.handle.net/2142/32405

[10] *MaGen Reference Manual*, Haverly Systems, Inc., Denville, NJ, 1977.

[11] *GAMMA 3.4 Programmer Reference*, Sperry Univac Computer Systems, St. Paul, MN, 1977.

[12] *MIPS III DATAFORM User Manual*, Ketron, Inc., Arlington, VA, 1975.

[13] R. Fourer, "Modeling languages versus matrix generators for linear programming," *ACM Trans. Math. Softw.*, vol. 9, no. 2, pp. 143–183, Jun. 1983. [Online]. Available: https://doi.org/10.1145/357456.357457

[14] A. Brooke, D. Kendrick, and A. Meeraus, *GAMS: A User's Guide*. Redwood City, CA: The Scientific Press, 1988.

[15] J. Bisschop, *AIMMS optimization modeling*, 2006.

[16] R. Fourer, D. M. Gay, and B. W. Kernighan, "A modeling language for mathematical programming," *Management Science*, vol. 36, no. 5, pp. 519–554, 1990.

[17] M. ApS, *MOSEK optimization suite 11.0*, 2026.

[18] ——, "MOSEK modeling cookbook," 2026.

[19] W. E. Hart, J.-P. Watson, and D. L. Woodruff, "Pyomo: modeling and solving mathematical programs in Python," *Mathematical Programming Computation*, vol. 3, no. 3, pp. 219–260, 2011.

[20] I. Dunning, J. Huchette, and M. Lubin, "JuMP: A modeling language for mathematical optimization," *SIAM Review*, vol. 59, no. 2, pp. 295–320, 2017.

[21] J. A. E. Andersson, J. Gillis, G. Horn, J. B. Rawlings, and M. Diehl, "CasADi – A software framework for nonlinear optimization and optimal control," *Mathematical Programming Computation*, vol. 11, no. 1, pp. 1–36, 2019.

[22] I. CVX Research, "CVX: Matlab software for disciplined convex programming, version 2.0," https://cvxr.com/cvx, Aug. 2012.

[23] A. Agrawal, R. Verschueren, S. Diamond, and S. Boyd, "A rewriting system for convex optimization problems," *Journal of Control and Decision*, vol. 5, no. 1, pp. 42–60, 2018.

[24] M. Udell, K. Mohan, D. Zeng, J. Hong, S. Diamond, and S. Boyd, "Convex optimization in Julia," in *2014 first workshop for high performance technical computing in dynamic languages*. IEEE, 2014, pp. 18–28.

[25] A. Agrawal, S. Diamond, and S. Boyd, "Disciplined geometric programming," *Optimization Letters*, vol. 13, pp. 961–976, 2019.

[26] A. Agrawal and S. Boyd, "Disciplined quasiconvex programming," *Optimization Letters*, vol. 14, pp. 1643–1657, 2020.

[27] A. Agrawal, B. Amos, S. Barratt, S. Boyd, S. Diamond, and Z. Kolter, "Differentiable convex optimization layers," in *Advances in Neural Information Processing Systems*, vol. 32, 2019.

[28] M. Lubin, O. Dowson, J. D. Garcia, J. Huchette, B. Legat, and J. P. Vielma, "Jump 1.0: Recent improvements to a modeling language for mathematical optimization," 2023. [Online]. Available: https://arxiv.org/abs/2206.03866

[29] B. Legat, O. Dowson, J. D. Garcia, and M. Lubin, "Mathoptinterface: a data structure for mathematical optimization problems," 2020. [Online]. Available: https://arxiv.org/abs/2002.03447

[30] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *nature*, vol. 521, no. 7553, pp. 436–444, 2015.

[31] L. Bottou, F. E. Curtis, and J. Nocedal, "Optimization methods for large-scale machine learning," *SIAM Review*, vol. 60, no. 2, pp. 223–311, 2018. [Online]. Available: https://epubs.siam.org/doi/10.1137/16M1080173

[32] Y. LeCun, "A theoretical framework for back-propagation," in *Proceedings of the 1988 Connectionist Models Summer School*, vol. 1. CMU, Pittsburgh, Pa, 1988, pp. 21–28.

[33] A. Griewank and A. Walther, *Evaluating Derivatives: Principles and Techniques of Algorithmic Differentiation*, 2nd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2008.

[34] P. C. Chakravarti, "Truncation error in interpolation and numerical differentiation," *Numer. Math.*, vol. 3, no. 1, p. 279–284, Dec. 1961. [Online]. Available: https://doi.org/10.1007/BF01386029

[35] P. Wolfe, "Checking the calculation of gradients," *ACM Trans. Math. Softw.*, vol. 8, no. 4, p. 337–343, Dec. 1982. [Online]. Available: https://doi.org/10.1145/356012.356013

[36] Wolfram Research, Inc., "Mathematica, Version 14.3," 2026, champaign, IL. [Online]. Available: https://www.wolfram.com/mathematica

[37] Maplesoft, a division of Waterloo Maple Inc., "Maple," Waterloo, Ontario. [Online]. Available: https://www.maplesoft.com

[38] A. Meurer, C. P. Smith, M. Paprocki, O. Čertík, S. B. Kirpichev, M. Rocklin, A. Kumar, S. Ivanov, J. K. Moore, S. Singh, T. Rathnayake, S. Vig, B. E. Granger,

R. P. Muller, F. Bonazzi, H. Gupta, S. Vats, F. Johansson, F. Pedregosa, M. J. Curry, A. R. Terrel, v. Roučka, A. Saboo, I. Fernando, S. Kulal, R. Cimrman, and A. Scopatz, "Sympy: symbolic computing in python," *PeerJ Computer Science*, vol. 3, p. e103, Jan. 2017. [Online]. Available: https://doi.org/10.7717/peerj-cs.103

[39] S. Laue, "On the equivalence of forward mode automatic differentiation and symbolic differentiation," *ArXiv*, vol. abs/1904.02990, 2019. [Online]. Available: https://api.semanticscholar.org/CorpusID:102350450

[40] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, "Automatic differentiation in machine learning: a survey," *J. Mach. Learn. Res.*, vol. 18, no. 1, p. 5595–5637, Jan. 2017.

[41] J. Bradbury, R. Frostig, P. Hawkins, M. J. Johnson, Y. Katariya, C. Leary, D. Maclaurin, G. Necula, A. Paszke, J. VanderPlas, S. Wanderman-Milne, and Q. Zhang, "JAX: composable transformations of Python+NumPy programs," 2018. [Online]. Available: http://github.com/jax-ml/jax

[42] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, *PyTorch: an imperative style, high-performance deep learning library.* Red Hook, NY, USA: Curran Associates Inc., 2019.

[43] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean, M. Devin, S. Ghemawat, G. Irving, M. Isard, M. Kudlur, J. Levenberg, R. Monga, S. Moore, D. G. Murray, B. Steiner, P. Tucker, V. Vasudevan, P. Warden, M. Wicke, Y. Yu, and X. Zheng, "Tensorflow: a system for large-scale machine learning," in *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI'16. USA: USENIX Association, 2016, p. 265–283.

[44] U. Naumann, *The Art of Differentiating Computer Programs: An Introduction to Algorithmic Differentiation*, ser. Software, Environments, and Tools. Philadelphia, PA: SIAM, 2011.

[45] P. Herholz, X. Tang, T. Schneider, S. Kamil, D. Panozzo, and O. Sorkine-Hornung, "Sparsity-specific code optimization using expression trees," *ACM Trans. Graph.*, vol. 41, no. 5, May 2022. [Online]. Available: https://doi.org/10.1145/3520484

[46] A. H. Gebremedhin, A. Tarafdar, A. Pothen, and A. Walther, "Efficient computation of sparse hessians using coloring and automatic differentiation," *INFORMS J. on Computing*, vol. 21, no. 2, p. 209–223, Apr. 2009. [Online]. Available: https://doi.org/10.1287/ijoc.1080.0286

[47] A. H. Gebremedhin, F. Manne, and A. Pothen, "What color is your Jacobian? Graph coloring for computing derivatives," *SIAM Review*, vol. 47, no. 4, pp. 629–705, 2005.

[48] B. W. Kernighan and D. M. Ritchie, *The C Programming Language*, 2nd ed. Prentice Hall, 1988.

[49] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2003.

[50] T. A. Davis, *Direct Methods for Sparse Linear Systems*. Philadelphia, PA: Society for Industrial and Applied Mathematics, 2006. [Online]. Available: https://epubs.siam.org/doi/book/10.1137/1.9780898718881

[51] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, İ. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, and SciPy 1.0 Contributors, "SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[52] I. S. Duff, A. M. Erisman, and J. K. Reid, *Direct Methods for Sparse Matrices*. Oxford, England: Oxford University Press, 1986.

[53] P. E. Gill, W. Murray, M. A. Saunders, and M. H. Wright, "Sparse matrix methods in optimization," *SIAM Journal on Scientific and Statistical Computing*, vol. 5, no. 3, pp. 562–589, 1984.